`CommentForm` instance. If the method wasn't GET, it must have been POST. In the POST case, the user submitted a form, and the data from the submission is contained in `request.POST`. You pass this data into your form's initializer to populate it.

The Form base class provides the `.is_valid()` method for validating the data sent by the user. The validation process is based on the fields in the form and their configuration. In the `CommentForm` case, the data is all text. As data submitted by HTTP is all text, there is very little validation done here. The fields are required, which the validator enforces, but if the user used the browser to submit the form, the browser would have enforced this anyhow.

Tricky, tricky users

Never trust your users. That last paragraph had an important caveat in it: "if the user used the browser to submit the form." Although well-behaved users will do this, your site openly accepts requests from HTTP. There are ways of issuing an HTTP POST without using the browser or, in the browser, using development tools that don't enforce the form's rules.

Always (it bears repeating: *always*) assume your users are malicious. Browser-side JavaScript validation is helpful to users, giving them a quick response on errors, but it should never be the only enforcement. Always validate your data in your views.

If the form is valid, you have a comment. Now what? Let's inform the admin through email. The `send_mail()` function in `django.core.mail` is a quick way to send an email using the currently configured email backend. In the previous chapter, you configured Django to print email messages to the console using the `console.EmailBackend` class. Using `send_mail()`, you can send a note with the contents of the comment. The note will be from the admin, and to the admin, you're sort of talking to yourself.

The `.is_valid()` method on a form sets up a dictionary in the form named `cleaned_data`. This dictionary has a key–value pair for each field named in the form and its contents. Using `cleaned_data`, you construct an email message and then send it. By default, the `send_mail()` function fails silently; this is not usually the best choice, so add `fail_silently=False` to the call, causing an exception if there is a problem. With the development server, this kind of problem shows up in the console as a `500` error, while in a production environment, this shows in your logs.

Once the form gets processed, you need to send the user somewhere new. A second view, called `comment_accepted()`, displays a thank-you note. This view can take advantage of the general.html template created in the previous chapter. To send the user to another view, you use the `redirect()` function from the `django.shortcuts` module. This function takes at least one argument which indicates where to redirect the user to. The argument can be a string containing a named URL path, a string containing a URL, or a reference to a view function. If you are using the name of a path or a view and the associated view takes arguments, you also pass those to `redirect()`.